

TodoFlow — Diagram Explanations

A companion guide to the technical diagrams referenced in Chapter Three (Methodology) of the TodoFlow project report. Each diagram below has a matching editable .drawio file in ProjectDocs/drawio/ — open it at <https://app.diagrams.net> (File > Open From > Device) to edit, relabel, or restyle it, then re-export as an image for the report if you make changes.

1. System Architecture Diagram

Files: [diagrams/fig_3_1_architecture.png](#) (image, embedded in the report) · [drawio/fig_3_1_architecture.drawio](#) (editable source)

What it shows:

TodoFlow's software architecture as five horizontal layers, stacked in the order a request travels through them, with a side panel calling out the concerns that cut across every layer instead of belonging to just one.

How to read it

- Presentation Layer (blue) — Blade templates, Bootstrap 5, and Livewire's browser-side JavaScript. This is what actually renders in the user's browser.
- Component Layer (green) — the Livewire PHP components (Dashboard, Tags, Trash, AlertCenter, the Admin components). Their job is narrow: validate form input, then hand off to a service.
- Service Layer (amber) — the nine dedicated service classes that hold every piece of business logic and authorisation logic in the application. This is the layer a Livewire component is never allowed to bypass.
- Domain Layer (purple) — Eloquent models and their authorisation policies, which the service layer calls into to actually read or write data.
- Persistence Layer (red) — the relational database itself (SQLite in development, MySQL in production).
- The double-headed arrows between layers represent a request going down (an action) and a response coming back up (the result, or a caught error).

- The side panel documents the TryAction trait, the dedicated services log channel, and the Laravel scheduler — three things that apply across every layer rather than living in one of them.

Why it matters for the report: this diagram is what section 3.5.4 and the testing discussion in section 4.2.1 both refer back to — it is the single picture that explains why a failure in one Livewire action cannot crash the whole page.

2. Data Flow Diagram — Level 0 (Context Diagram)

Files: diagrams/fig_3_2_dfd_level0.png (image, embedded in the report) · drawio/fig_3_2_dfd_level0.drawio (editable source)

What it shows: TodoFlow treated as a single black-box process, with every external entity that sends data into it or receives data out of it drawn around the outside. This is the highest, simplest possible view of the system — deliberately with no internal detail.

How to read it

- The circle in the middle is the whole TodoFlow system (numbered 0.0, the standard DFD convention for the top-level process).
- User and Administrator (left) are the two human actors who send data in (todo/tag/list data, or user-management commands) and receive data out (task lists, alerts, reports).
- Task Scheduler (right) is not a person — it is the automated cron-driven process that triggers alert generation every five minutes and receives back the list of reminder/overdue data it needs to act on.
- Every arrow is labelled with the data that flows along it, not with a technical detail like a table or endpoint name — that is what Level 1 is for.

3. Data Flow Diagram — Level 1

Files: diagrams/fig_3_3_dfd_level1.png (image, embedded in the report) · drawio/fig_3_3_dfd_level1.drawio (editable source)

What it shows: the single process from the Level 0 diagram broken open into its six major internal processes, plus the four data stores those processes actually read from and write to. This is the diagram that maps roughly one-to-one onto the service layer described in the architecture diagram.

How to read it

- Circles are processes: 1.0 Authenticate User, 2.0 Manage Todos, 3.0 Manage Tags & Lists, 4.0 Manage Trash, 5.0 Generate & View Alerts, and 6.0 Administer Users.
- The open-ended rectangles (data stores) are D1 Users, D2 Todos/Lists/Tags, D3 Notifications, and D4 Sync Logs/Devices — these correspond directly to tables in the ER diagram and database schema.
- Arrows from User/Administrator into a process represent a request; arrows from a process into a store represent that request being read or written to the database.
- Notice process 4.0 (Manage Trash) writes to D2 with a "soft delete" label rather than a plain delete — this is the diagram-level representation of TodoFlow never destroying a record outright.

4. Use Case Diagram

Files: diagrams/fig_3_4_use_case.png (image, embedded in the report) · drawio/fig_3_4_use_case.drawio (editable source)

What it shows: every distinct action ("use case") a User or an Administrator can perform in TodoFlow, grouped inside the system boundary, with each actor connected to the set of use cases available to them.

How to read it

- The stick figures are the two actors: User (left) and Administrator (right).
- Each oval inside the rectangle labelled "TodoFlow System" is one use case — one thing an actor can do, phrased as a verb phrase (e.g. "Manage Todo Lists", "View Trash").
- The blue ovals are available to any authenticated User. The purple ovals (bottom right) are Administrator-only.

- Because an Administrator account is still a User of their own todos, the diagram treats Administrator as inheriting every User use case, plus the purple ones — rather than drawing twenty separate connecting lines, a single labelled connector to the boundary represents that whole relationship, which keeps the diagram readable.

5. Entity Relationship Diagram (ERD)

Files: [diagrams/fig_3_5_erd.png](#) (image, embedded in the report) · [drawio/fig_3_5_erd.drawio](#) (editable source)

What it shows: every table in TodoFlow's database, the columns each one holds, and how the tables relate to one another through foreign keys. This is the most detailed and most frequently referenced diagram in the report.

How to read it

- Each blue box is one database table (entity): its dark header names the table, and the list beneath it is every column in that table, including its primary key (PK) and any foreign keys (FK).
- Lines between boxes are relationships, labelled with their cardinality — almost everything in TodoFlow is "1:M" (one-to-many): one user has many todos, one todo list has many todos, and so on.
- tags has a nullable user_id — the one deliberate exception in the schema. A tag with no owner is a system tag, shared by every user; this is called out explicitly on the relationship line ("1:M (nullable)").
- notifications relates to users through a polymorphic relationship (labelled "1:M (poly)") rather than a normal foreign key, because Laravel's notification system is written to be reusable against any kind of "notifiable" model, not just User.
- Every table except sync_logs carries a deleted_at column — the ERD is where the soft-delete design decision discussed in Chapter Four is most visible at a glance.

6. Database Schema Diagram

Files: [diagrams/fig_3_6_schema.png](#) (image, embedded in the report) · [drawio/fig_3_6_schema.drawio](#) (editable source)

What it shows: the same nine tables as the ER diagram, but as a flat reference list rather than a relationship graph — every table name on the left, every one of its columns spelled out on the right. Where the ER diagram answers "how do these tables relate?", this diagram answers "exactly what is in this table?"

How to read it

- Read it top to bottom as a reference sheet, not as a diagram to trace arrows through — there are no relationship lines here on purpose.
- The `version` and `client_updated_at` columns present on almost every table are not used by anything in the current system — they are reserved for the future multi-device synchronisation layer discussed in section 5.3.1, so a future developer extending `TodoFlow` does not need to alter the schema to add it.

7. Iterative Development Model Diagram

Files: [diagrams/fig_3_7_methodology.png](#) (image, embedded in the report) · [drawio/fig_3_7_methodology.drawio](#) (editable source)

What it shows: the six-step cycle described in section 3.2.1, arranged as a loop rather than a straight line, because the whole point of the iterative model `TodoFlow` was built with is that the process repeats — each feature area went through this same cycle independently rather than the project going through it once from end to end.

How to read it

- Follow the arrows clockwise starting from "1. Requirement Planning" at the top: Design, Development, Testing, Review & Feedback, and Deployment/Next Iteration.
- The arrow from step 6 back to step 1 is the important part of this diagram — it is what makes the model iterative rather than a one-directional waterfall. Each of `TodoFlow`'s major feature areas (todo management, tag management, the trash system, the alert system, the admin panel) went around this loop as its own cycle.